Configuration Management for Services

same build, different config

Watch the 4-min video

In one line: keep config out of the code so one build runs everywhere; store it centrally, versioned and per environment; keep secrets in a secret store referenced from config; and change values (and feature flags) at run time, gradually and reversibly, without redeploying.

1 A two-second timeout, forty instances

A timeout must change from 2 s to 5 s. It is hard-coded, so you rebuild, test and redeploy forty instances. Meanwhile staging and production drift apart because someone edited a file on a server. Good configuration management makes such changes safe, fast and visible.

2 Config out of the code

The **twelve-factor app** rule: store config in the environment, separate from code. Config is everything that varies between deployments: URLs, timeouts, pool sizes, feature switches, credentials. The **same build artefact** runs in dev, staging and production; only the config differs. That keeps environments comparable and makes a tested build the one you ship.

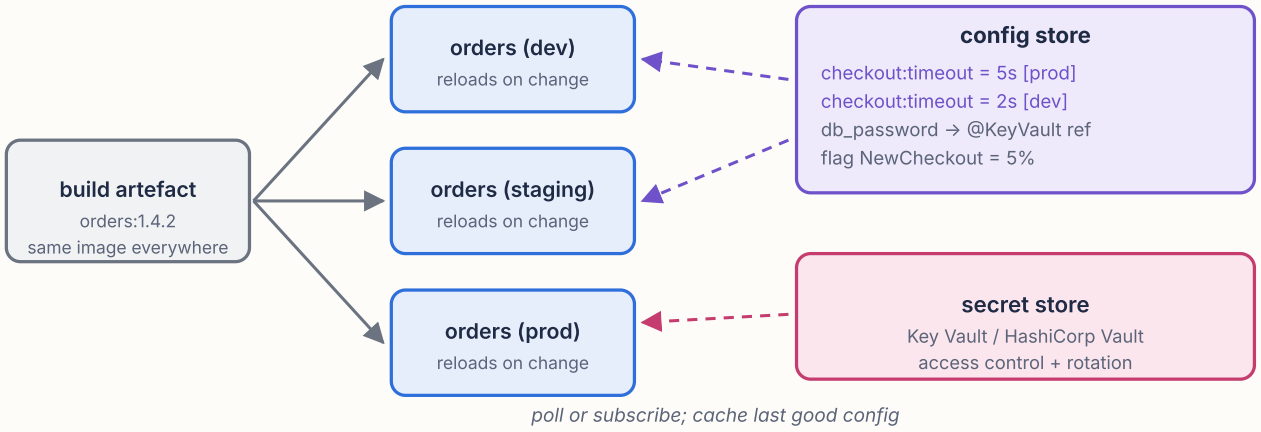


Figure 1. One image, three environments. Settings come from a labelled store; secrets are only referenced, never copied into config.

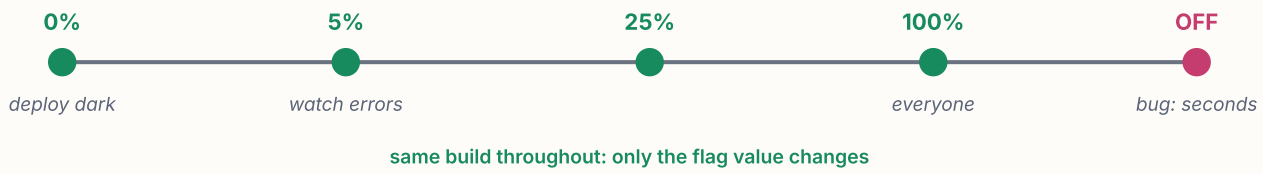
3 Where config lives

Option	Good for	Limits
Environment variables	few settings, set by the platform	change needs a restart; no audit trail
Config files in Git, applied by pipeline	reviewed, versioned changes (config as code)	change needs a pipeline run
Central store: Azure App Configuration, AWS AppConfig, Spring Cloud Config, Consul	many services; labels per environment; history and audit; dynamic reload	another dependency: cache values in case it's down
Secret store: Azure Key Vault, HashiCorp Vault, AWS Secrets Manager	passwords, API keys, connection strings	strict access policies; rotate regularly

Hierarchical keys such as `checkout:timeout` keep related settings together. Config may hold a **reference** to a secret (Azure App Configuration supports Key Vault references) but never the secret itself, and secrets never go into Git, even in a private repository (lesson 48).

4 Dynamic config and feature flags

To change values without redeploying, services **poll** the store or **subscribe** to change notifications and reload in memory. Validate new values (a timeout of 0 is a bug), roll changes out gradually like code, and cache the **last known good** config so a service can start even when the store is down. **Feature flags** are dynamic config for behaviour (lesson 76): turn a feature on for 5% of users, then everyone, or off in seconds.



same build throughout: only the flag value changes

Figure 2. A flag rollout: exposure grows step by step, and the kill switch needs no deploy.

CONFIG DRIFT AND BAD PUSHES

- **Drift:** environments or instances end up different through manual edits. Keep config as code, apply it by pipeline, alert on differences.
- **Bad pushes:** a wrong value can cause an outage like bad code. Validate, stage the rollout, and keep one-click rollback to the previous version.
- **Undocumented settings:** record each key's meaning, default and owner.
- **Stale flags:** remove flags once a feature is fully released.

WORKED EXAMPLE: APP CONFIGURATION, KEY VAULT AND A FLAG

Services read settings from Azure App Configuration with a label per environment (`checkout:timeout = 5s` under `prod`) and the database password through a Key Vault reference. A new checkout flow ships behind the flag `NewCheckout`. The team enables it for 5% of users, watches error rates and conversion, then raises it to 100%. When a bug appears, they switch the flag off in seconds, with no redeploy, then fix it calmly.

5 Key terms

Twelve-factor config: config in the environment, not in code

Config store: central, versioned, labelled settings service

Dynamic config: values reloaded at run time without redeploy

Environment parity: one artefact; only config differs

Secret reference: pointer from config to a secret store entry

Config drift: environments silently diverging

6 Interview questions

Q1. What does "keep config out of the code" mean? BASIC

Everything that varies by environment (URLs, timeouts, flags, credentials) is supplied at deploy or run time, so the same build artefact runs everywhere.

Q2. Where should secrets go? BASIC

In a dedicated secret store with access control and rotation, referenced from config; never in code, config files or Git.

Q3. How do you change config without redeploying? SCENARIO

Keep it in a central store, have services poll or subscribe for changes and reload in memory, roll changes out gradually, and use feature flags for behaviour changes.

Q4. A new checkout flow is risky. How do you release it? SCENARIO

Ship it dark behind a feature flag, enable it for a small percentage while watching metrics, raise it step by step, and turn it off instantly if it fails.

Q5. The config store is down when a service starts. What happens? TRICKY

It should start from a cached last-known-good config or safe defaults rather than fail, and alert; otherwise the store becomes a single point of failure.

Q6. Staging passed but production broke on the same build. A likely cause? TRICKY

Config drift: production settings differ from staging; keep config as code applied by pipeline and compare environments automatically.

Next: SD-M-001 · Authentication vs Authorization backendroadmap.com